Reinforcement Learning for Lean Proofs with a Small Language Model

Midterm report: formulation, experiments, and findings

Manas Ravulapalli

13 October 2026

Abstract

This report asks whether reinforcement learning (RL) can teach a small language model, `Qwen2.5-0.5B-Instruct`, to write Lean 4 proofs of elementary number theory. We formulate proof writing as a Markov decision process (MDP) with a reward of 1 only for a complete proof that Lean accepts. Most experiments failed for one main reason: when the model never succeeds, the RL update is exactly zero. RL helped only on tasks that the model could already solve some of the time. We also found ways to obtain the reward without proof skill, and several engineering faults.

Main findings

1. RL needs the model to succeed some of the time. With no success, the gradient is zero (Section 2.2).
2. The first barrier was the Lean language, not the mathematics (Section 4.1).
3. A reward that does not check correctness can increase while accuracy decreases (Section 4.3).
4. RL helped twice, both times on tasks that the model could already solve some of the time (Sections 4.4 and 4.6).
5. A Lean acceptance does not always show skill: the checker had faults, and some statements are trivial (Section 5).
6. The next run uses a corrected environment, a curriculum ordered by proof structure, and a held-out panel cleaned of tasks that restate training statements (Section 7).

1 The problem

We want a small language model that writes proofs that the proof assistant Lean 4 can check. The topic is school and early-university number theory: parity, divisibility, gcd and lcm, primes, and congruences. The midterm goal was a model that produces some complete proofs that Lean accepts. The goal for the end of the semester is proof strategy on competition-style problems.

Motivation.

1. **Efficiency.** We want the most proofs for a fixed compute budget. We study the *curriculum* (which tasks, in which order) and the *distribution of reward* over proof steps.
2. **Checked learning.** A model that learns without a checker can learn false statements. Lean should check the model while it learns, not only at the end.
3. **One model for informal and formal steps.** The model writes a short informal step and then its Lean step. A separate translator would add errors and compute.

4. **No human graders.** Lean gives an exact reward. This setting is called RL with verifiable rewards (RLVR). The reward is *sparse*: a signal comes only from a complete proof.

What Lean does. A Lean theorem has a statement and a proof. We write the proof as a list of *tactics*. Each tactic changes the *goals*, the facts that remain to be proved. When no goals remain, the proof is complete. Table 1 shows a small example that we wrote to explain the idea, and Figure 1 shows when Lean accepts a proof.

Table 1: A tactic proof that $a \mid b$ and $b \mid c$ imply $a \mid c$ (here $a \mid b$ means $b = ak$). This example explains the idea; it is not a measured proof.

Step	Goal before the step	Tactic (the action)	Effect
1	$a \mid c$, given $a \mid b$, $b \mid c$	`obtain ⟨k, hk⟩ := h1`	Adds $b = ak$
2	$a \mid c$, given $b = ak$, $b \mid c$	`obtain ⟨l, hl⟩ := h2`	Adds $c = bl$
3	$a \mid c$, given $b = ak$, $c = bl$	`exact ⟨k * l, by rw [hl, hk, Nat.mul_assoc]⟩`	$c = (ak)l = a(kl)$. No goals remain

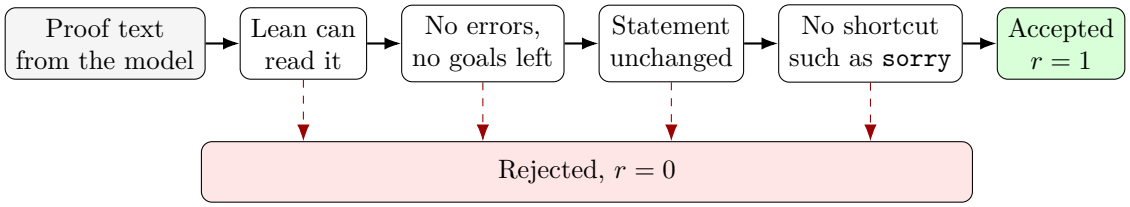


Figure 1: When Lean accepts a proof. Every check must pass. One wrong character or one invented lemma name causes a rejection.

Setup. Model: `Qwen/Qwen2.5-0.5B-Instruct` [1], with 494,032,768 parameters. Checker: Lean `v4.10.0-rc1` with one fixed mathlib version. Compute: one rented NVIDIA L40S GPU, and a shared CPU server.

2 Formulation as an MDP

An MDP describes an agent that acts step by step [3]. At step t, the agent observes a state s_t, selects an action a_t, receives a reward r_t, and moves to a state s_{t+1}. Here the agent (the *policy* π_θ) is the language model, and the environment is Lean. We used two formulations, which differ only in the size of one action (Figure 2).

Table 2 defines Formulation B, and Figure 3 shows it as a graph.

Version 1 has three defects. The model does not see the remaining steps, the theorem, or the proof so far. With a step limit that the model cannot see, the process is not exactly Markov. One error ends the episode. Section 7 corrects these defects.

2.1 Learning algorithm

We maximize the expected return $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t \gamma^t r_t\right]$. The only reward is at the end, so the return of an episode τ is $R(\tau) \in \{0, 1\}$. For one theorem, we sample G episodes and use the REINFORCE

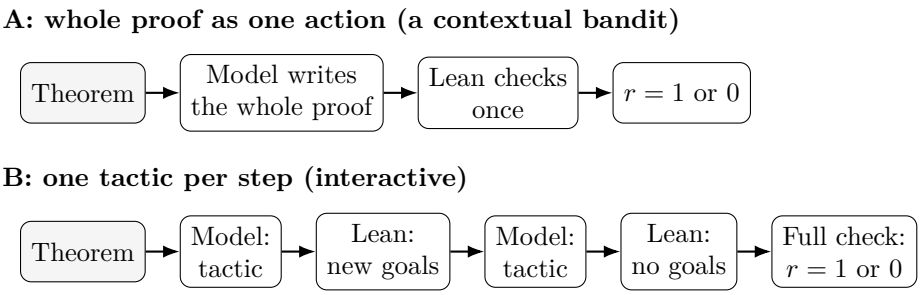


Figure 2: The two formulations. In A, the model receives no feedback while it writes (Experiments 4 to 6, 20 to 22). In B, Lean runs each tactic first (Experiments 19 and 23).

Table 2: The interactive MDP (environment version 1).

Part	Definition
State s_t	The current Lean goals. At $t = 0$, the theorem. In an *assisted* task, part of a known proof is applied first
Action a_t	A short tactic block (at most 64 tokens in the main run)
Transition	Lean runs the block. If it succeeds, the new goals are s_{t+1}
End of episode	No goals remain (then Lean checks the full proof), Lean reports an error, or the step budget is used (4 actions)
Reward	1 at the end if Lean accepts the full proof, otherwise 0
Discount	$\gamma = 1$

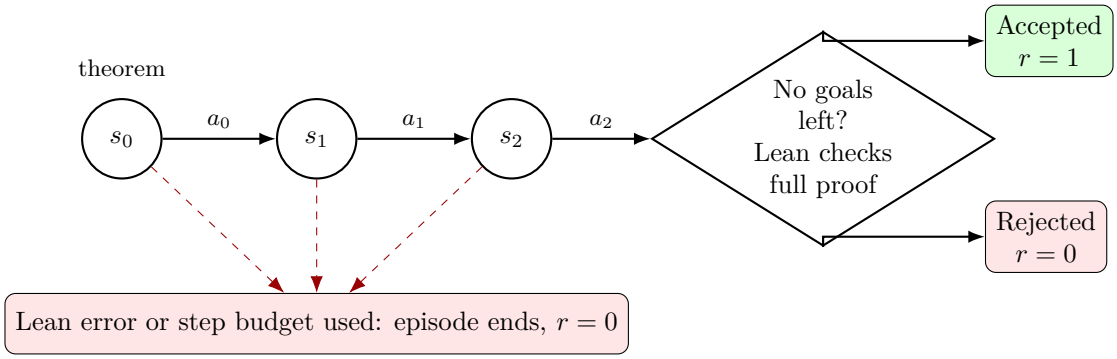


Figure 3: Formulation B for the proof of Table 1. Solid arrows: the tactic succeeds. Dashed arrows: an error ends the episode with reward 0.

estimator [4]

$$\nabla_\theta J(\theta) \approx \frac{1}{G} \sum_{i=1}^{G} A_i \, \nabla_\theta \log \pi_\theta(\tau_i). \tag{1}$$

The *advantage* A_i compares attempt i with the other attempts. The interactive runs use the leave-one-out baseline (RLOO) [5], and the whole-proof runs use GRPO [6, 7]:

$$A_i^{\text{RLOO}} = R_i - \frac{1}{G-1} \sum_{j \neq i} R_j, \qquad A_i^{\text{GRPO}} = \frac{R_i - \bar{R}}{s_R + \varepsilon}, \tag{2}$$

where $\bar{R}$ and s_R are the mean and standard deviation of the G rewards.

2.2 Main consequence: no success, no learning

If all G attempts fail, every $R_i = 0$, so every $A_i = 0$, and the update in Equation (1) is exactly zero. The same is true when all attempts succeed. The model learns only from *mixed groups*, which contain at least one success and one failure (Figure 4).

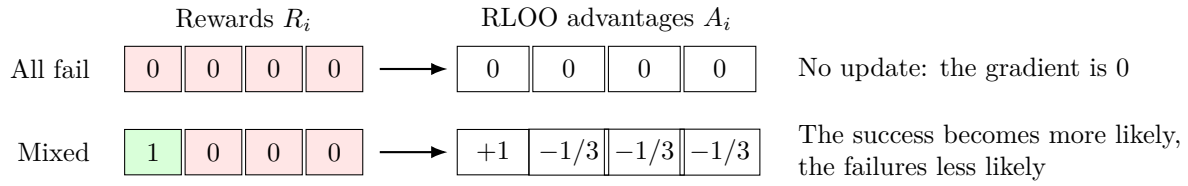


Figure 4: A group needs both a success and a failure ($G = 4$).

If the model succeeds with probability p per attempt, a group of G independent attempts is mixed with probability

$$P_{\text{mix}}(p, G) = 1 - p^G - (1-p)^G. \tag{3}$$

Figure 5 shows that this probability is small when p is small. This equation explains most of our failures.

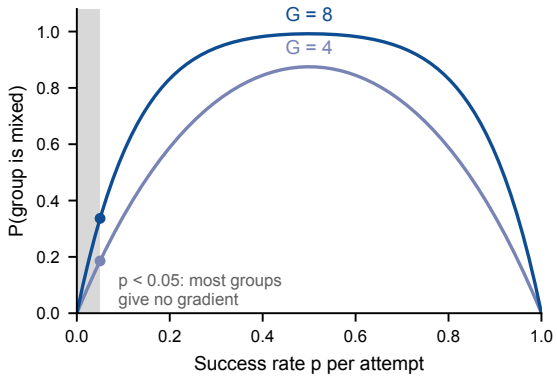


Figure 5: Probability that a group is mixed (Equation 3). At $p = 0.05$ it is 0.185 for $G = 4$ and 0.337 for $G = 8$.

2.3 A second reward: RLP

Some experiments used the RLP reward [8] instead of Lean. For a question x, reasoning c, and known answer y, the reward is $\log p_\theta(y \mid x, c) - \log p_{\bar{\theta}}(y \mid x)$, where $p_{\bar{\theta}}$ is a slowly updated copy of the model. The reward is high when the reasoning makes the correct answer more probable. It does not check whether the reasoning is correct (Figure 6). We call it a *heuristic* reward.

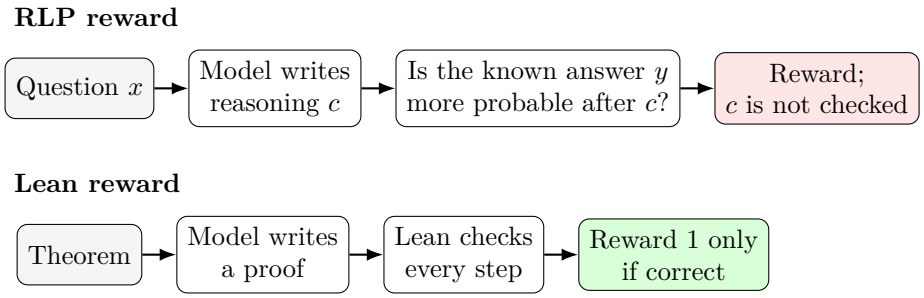


Figure 6: The two rewards. RLP rewards reasoning that makes a known answer more probable, correct or not. Lean rewards only a correct proof.

Measures. *Accepted attempts* counts sampled proofs that Lean accepts, for example 26/48. *Theorem coverage* counts theorems with at least one accepted attempt. One standard tactic proves 28 of the 420 validation statements without a model, so the main validation set is the other 392 (*non-trivial*) statements.

3 Data, training stages, and curricula

Table 3: Data sources and their use.

Source	Content	Use
LeanDojo Benchmark 4 [2]	Theorems and tactic proofs from mathlib; number-theory subset of 706 training, 17 validation, 15 test theorems	Supervised training; short proofs for interactive RL
Lean Workbook [9]	Competition-style problems as Lean statements	Pool of 4,306 number-theory statements that Lean checks
Number-theory SFT set	5,812 examples: Lean-accepted proofs, mathlib syntax examples, proof variants	Second supervised stage
Hand-written Lean tasks	Logic, arithmetic, divisibility, induction	Experiments 5, 6; DEV of Experiment 19
Hand-written arithmetic tasks	448 questions on gcd, inverses, primes, congruences	RLP (Experiment 9)
Open textbook (Judson)	Chapters of *Abstract Algebra: Theory and Applications*	RLP (Experiment 7)
Curriculum templates	192 training tasks in 5 stages and 152 held-out tasks, all checked by Lean	Final run (Section 7)

Table 3 lists the data. Figure 7 shows how we keep test data out of training. Renamed copies of one problem always go to the same split, and we never train on the DEV set.

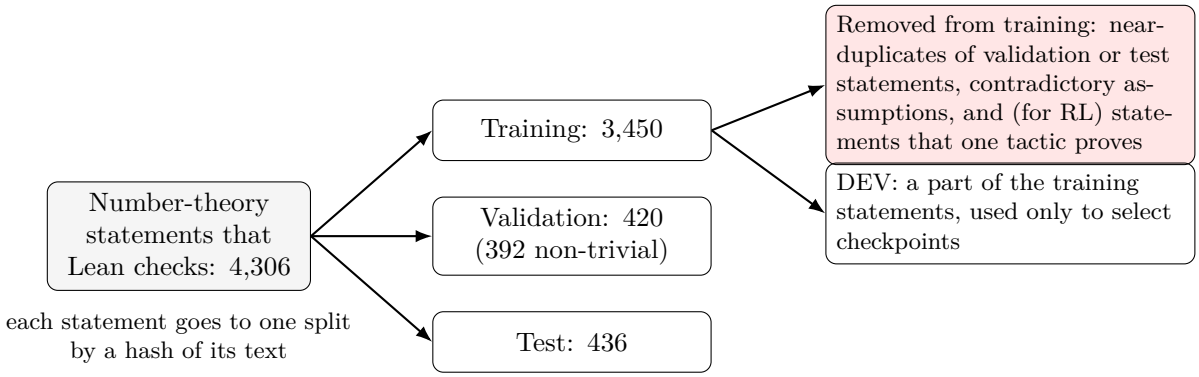


Figure 7: Data splits. Two statements are near-duplicates when their token sets overlap by at least 85%.

Training stages. All models come from one chain of checkpoints (Figure 8). SFT v2 is one pass over Lean proofs from the LeanDojo training files (4,526 steps of 64 examples), without any file that contains a validation or test theorem. Common SFT is a second supervised stage on the number-theory SFT set. Supervised fine-tuning (SFT) copies existing proofs. RL learns from the model's own attempts, so some attempts must succeed.

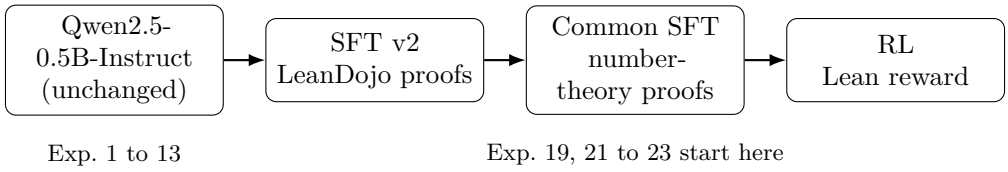


Figure 8: The training chain.

Curricula. RL needs tasks that the model can solve some of the time. A *curriculum* starts with easier versions of a task. We made tasks easier in three ways: we supplied knowledge in the prompt, we supplied part of the proof (a *reverse curriculum* [10], Figure 9), or we asked for one step instead of a whole proof. Most known proofs are short: 85% of the linear tactic proofs in our LeanDojo pool have one or two tactics, and only 24 have five or more. Therefore a reverse curriculum on these proofs has few levels. To test whether *order* matters, an *ordered* arm and a *shuffled* arm receive the same tasks the same number of times. Table 4 lists the curriculum of each experiment.

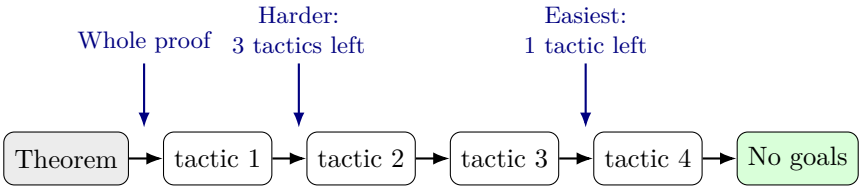


Figure 9: A reverse curriculum on one known proof. Each blue arrow is a start point; the earlier tactics are given. Training moves the start point toward the theorem.

Table 4: Curricula by experiment.

Exp.	Tasks	Order and support	Comparison
4	LeanDojo number-theory theorems	Grouped by topic	Random order
5	Hand-written Lean tasks	Logic, arithmetic, divisibility, induction; support decreases	Random order
6	Theorems with part of the proof given	Missing expression, then tactic, then body	Random order
9	448 arithmetic tasks	Prerequisites first, with replay	None
13	72 tasks in 5 versions	Support decreases from full information to none	Shuffled order
20	Number-theory proofs	Short proof endings, longer endings, whole proofs	SFT parent
19	3 practice proof endings	Fixed mixture	SFT parent
23	22 Lean Workbook theorems	Structure, lemma use, combined steps, whole proof	Shuffled order
Final	S0 to S4 templates	Identity, one lemma, two steps, chain of subgoals, 3 to 6 steps	Shuffled; adaptive

4 Experiments

Figure 10 shows all experiments in order, with the lesson that led to each next experiment. A cause is *tested* when an experiment could have shown it to be false; otherwise it is a *candidate*.

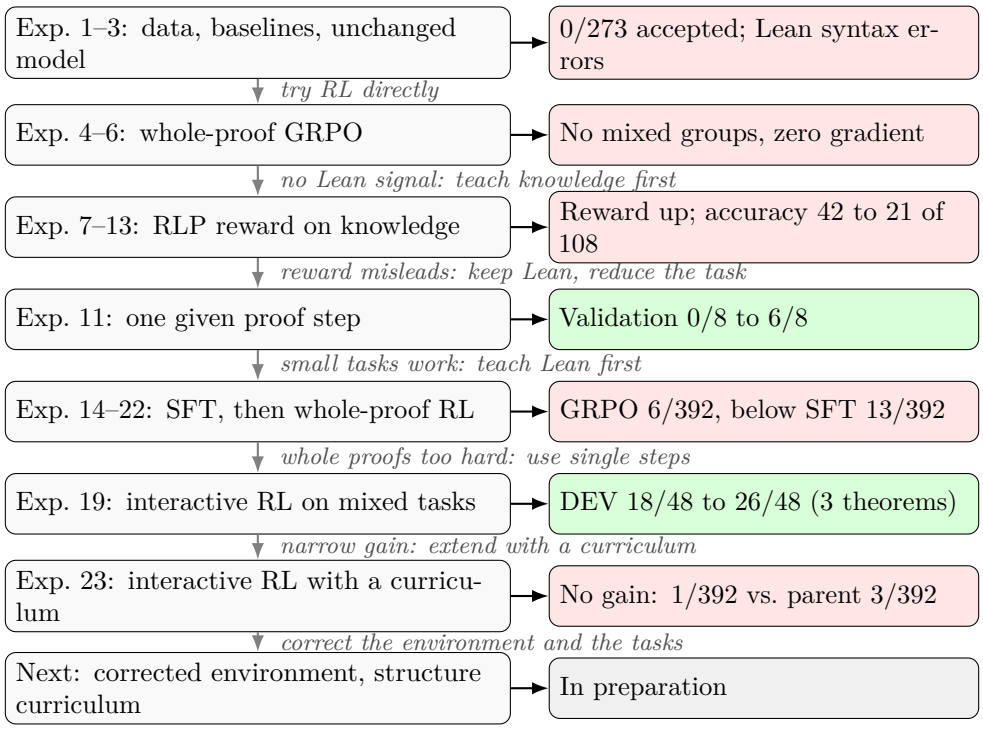


Figure 10: The sequence of experiments. Red: failure. Green: RL gain. Grey: in preparation.

4.1 Data, baseline, and the unchanged model (Experiments 1 to 3)

Experiment 1 selected a number-theory subset of LeanDojo. Its authors report that the random split overestimates performance [2], so we later built stricter splits (Figure 7). Experiment 2 tested the base

model on school mathematics: 9 of 24 GSM8K and 3 of 20 MATH number-theory problems. A small fine-tune lowered the held-out loss, but the scores became 8 of 24 and 3 of 20. A lower loss did not mean better answers.

Experiment 3 asked the unchanged model to prove 21 statements. Lean accepted 0 of 63 local attempts and 0 of 273 GPU attempts. Figure 11 shows that no attempt reached the mathematics. A pattern match found old Lean 3 syntax in 33 of the 63 attempts. In 34 of the 63, the model wrote a new theorem when we asked only for a proof. The first barrier is the language (*tested*: the classification reads Lean's error messages).

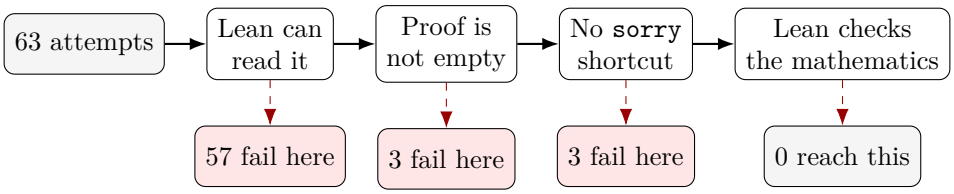


Figure 11: Where the 63 attempts of Experiment 3 failed.

4.2 Whole-proof RL from the unchanged model (Experiments 4 to 6)

We ran GRPO in Formulation A with $G = 4$. In Experiment 4, Lean accepted no proof, no group was mixed, and the gradient was zero for all 32 steps. Validation stayed at 0 of 8 theorems. In Experiment 5, easier hand-written tasks gave 0 of 256 accepted proofs, so we did not train. In Experiment 6, with one expression missing from a given proof, 2 of 64 attempts passed, but only one theorem gave mixed groups, and held-out attempts stayed at 0 of 16.

Why it failed (*tested*): every group had reward 0, so every update was 0 (Section 2.2). Therefore Experiment 4 gives no information about curriculum order.

4.3 RL with a reward that does not check correctness (Experiments 7 to 10, 12, 13)

Lean gave no signal, so we tried to teach knowledge first with the RLP reward.

- Experiment 7 (textbook text): held-out accuracy was 11/48 before and after.
- Experiment 8: only 8,557,971 of 494,032,768 weights had changed, because low-precision (BF16) rounding discards small updates. With full precision, accuracy stayed at 11/48.
- Experiment 9 (448 arithmetic tasks): the reward increased, but held-out accuracy decreased from 42/108 to 21/108 (Figure 12). Answers without a readable number increased from 17 to 69, and answers at the length limit from 15 to 36.
- Experiment 10: an early checkpoint improved at recognizing concepts (7 to 14 of 32), not at using them. Two AI graders disagreed on 44 of 321 answers.
- Experiment 12: all 612 numeric answers in the data were correct, but 160 of 448 rules omitted a condition (for example, $\gcd(a, n) = 1$ for a modular inverse). A constant answer solved every proof-by-contradiction item.
- Experiment 13 (decreasing support): 0 of 18 answers were fully correct at all five checkpoints.

Why it failed: the reward pays for reasoning that makes the answer more probable, not for correct reasoning. This is *reward hacking*. In Experiment 9, 17 incorrect answers received a positive advantage. A deliberately false line of reasoning increased the reward in both inspected examples. A *candidate*

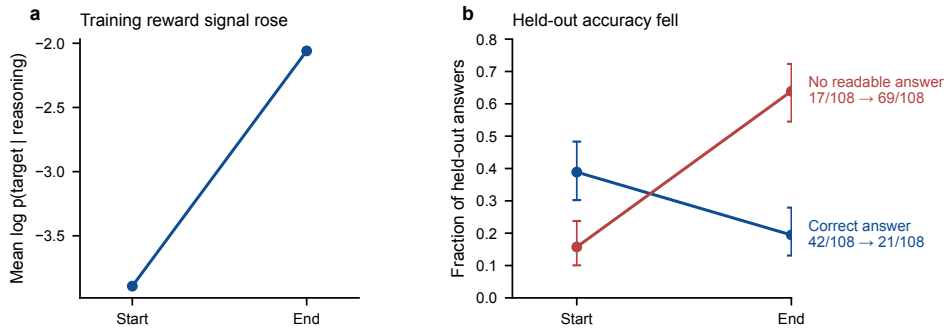


Figure 12: Experiment 9. (a) The quantity that the RLP reward pays for increased. (b) Held-out answers, $n = 108$. Error bars: 95% Wilson score intervals.

reason for the accuracy decrease is that training supplied the start of the answer format, but the test did not.

4.4 A smaller task (Experiment 11)

We reduced the task to one proof step with the mathematics supplied (Figure 13), and ran GRPO with $G = 8$. Validation accuracy increased from 0/8 to 6/8, and held-out accuracy from 0/4 to 2/4. It worked because some groups were mixed. The sets are small, and Lean checked only the expression: only 2 of 8 explanations had no mathematical error.

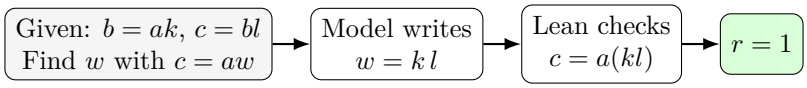


Figure 13: The one-step task of Experiment 11. Lean checks the expression, not the explanation.

4.5 Supervised training, then whole-proof RL (Experiments 14 to 22)

After SFT v2, the model solved 0 of the 392 non-trivial validation statements (2 of all 420). On 200 training statements it proved 0 in 1,600 attempts. The most frequent first errors were an invented lemma name (582), a failed rewrite (329), and the length limit (253). The barrier moved from syntax to library knowledge.

Experiment 20 added a common SFT stage, then alternated SFT with whole-proof GRPO in one branch and with RLP in another (Figure 14). Common SFT increased the count from 0 to 13. The GRPO branch ended at 6, below its parent, and the RLP branch at 15. The GRPO model wrote the single tactic `simp_all` in 1,637 of 3,360 samples; the parent did so in 62.

Experiment 21 used rejection-sampling fine-tuning (RFT: train on sampled proofs that Lean accepts [11]), then GRPO. On DEV (64 attempts per model), no model had an accepted proof. The first errors of the final model were proof construction (43), unknown names or types (17), syntax (3), and timeout (1). A parser fault rejected every action in a code block, so its step-by-step pilot made no update. Experiment 22 prepared PPO, but the GPU run failed before RL started.

Why it failed (*candidates*): the low success rate gives few mixed groups; the GRPO model concentrated on `simp_all`, which closes only easy goals. The branches also had extra SFT stages and different data, so the comparison does not isolate RL.

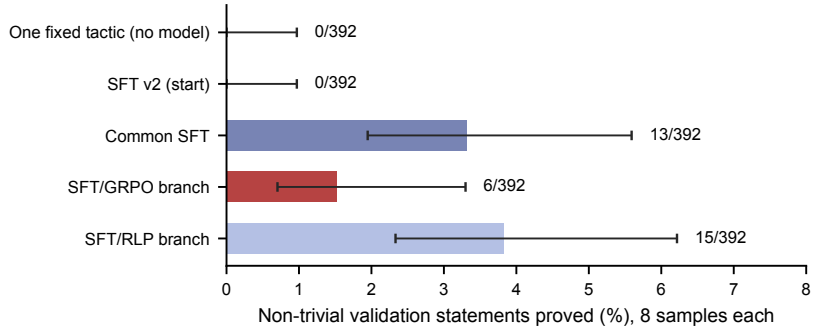


Figure 14: Experiment 20: non-trivial validation statements with an accepted proof in 8 attempts, $n = 392$. Error bars: 95% Wilson score intervals.

4.6 Step-by-step RL: the clear gain (Experiment 19)

We used Formulation B from common SFT, with REINFORCE, RLOO, $G = 4$, and at most 4 actions. We trained only on three practice tasks that the model solved some of the time. The success rule, fixed before the run, required more accepted DEV attempts than the parent over two seeds and no lost DEV theorem. The rule selected the RL model (Figure 15). It proved 7 of 32 original theorems without help (parent: 4 of 32), and all 7 proofs passed a new Lean check. An earlier run of this type lost one DEV theorem, so the rule kept its parent. Not every theorem improved: `Nat.not_even_one` decreased from 4/4 to 2/4.

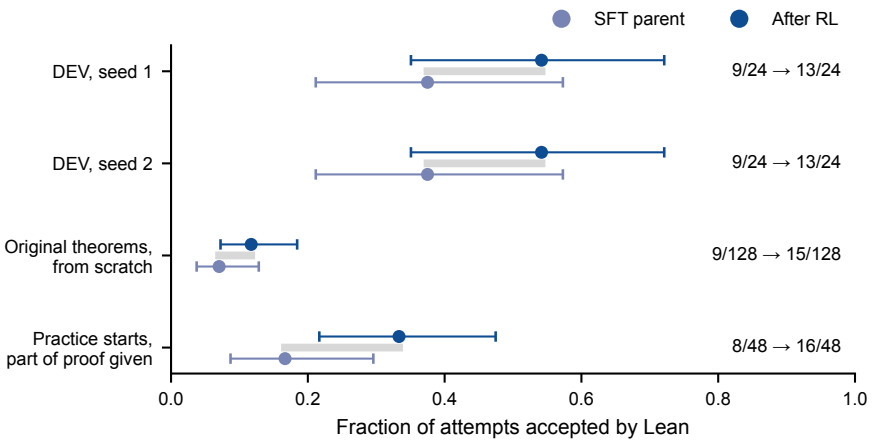


Figure 15: Experiment 19: SFT parent and RL model on the same tasks and seeds. Error bars: 95% Wilson score intervals, which treat correlated attempts as independent and so understate the uncertainty.

Limits: DEV has only 3 theorems, there is one training seed, and the 32 original theorems are from the training domain. RL made the model more reliable on proofs that it could already find; it did not show that RL teaches new proofs.

4.7 Step-by-step RL with a curriculum (Experiment 23)

On 22 broader theorems, a short warm-start first increased whole-proof success on 16 training tasks from 0 to 13. Then an *ordered* arm (answer given, one lemma, combined steps, whole proof) and a

shuffled arm trained on the same tasks. Neither arm improved on the parent. On broader DEV, all three models had 0 of 64. On easy retention, the parent had 24/48, ordered 20/48, and shuffled 21/48. On non-trivial validation, the parent proved 3 of 392 and ordered RL 1 of 392; one of the parent's accepted statements has the empty set as an answer. The ordered model made more valid first moves than the parent (88 against 40), but it completed no proof in its first move (Figure 16).

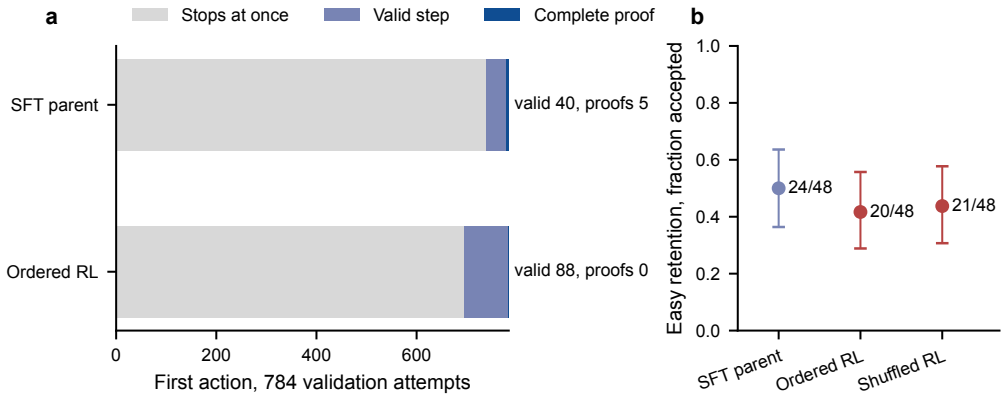


Figure 16: Experiment 23. (a) First action on the same 784 validation attempts. (b) Easy retention, $n = 48$. Error bars: 95% Wilson score intervals.

Why it failed (four *candidates*, not yet tested):

1. A valid step is not progress. The model learned valid steps, not complete proofs.
2. The curriculum had gaps: the lemma stage covered 9 of 22 theorems, the combined stage 11 of 22, and early tasks contained the answer.
3. For one Lean error type, the interface lost the message and gave reward 0, while ordinary errors received a penalty. A failure with a lost message therefore scored better (186 actions).
4. The environment did not show the theorem or the remaining steps, and one error ended the episode.

One common explanation is not supported: most repeated-state failures occurred without `simp_all`.

5 Ways to obtain the reward without proof skill

We searched for every way to obtain the Lean reward without real proof skill and found six types (Figure 17).

The checker attacks included false "no axioms" messages, a replaced axiom report, and a command after the proof. A scan of 10,869 accepted proofs found no attack in real runs. Some statements are also formalized incorrectly; for example, a divisibility statement about $(5 + \sqrt{35})^{2n-1}$ is about real numbers, where it is trivially true. Before we report an RL gain, we therefore separate trivial, contradictory, near-duplicate, and new statements, and report the new statements.

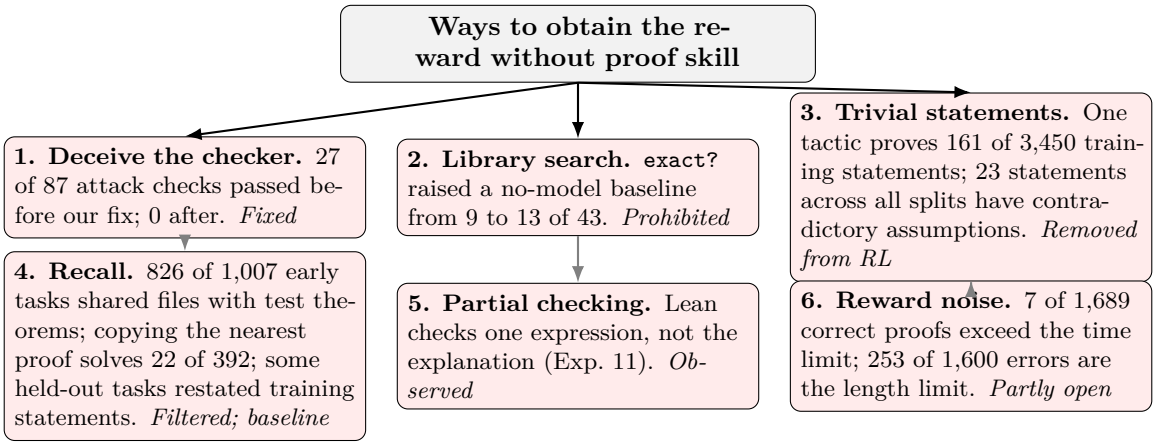


Figure 17: Six types of reward without real proof skill, with measurements and status.

6 Engineering faults

Table 5: Engineering faults that we found.

Fault	Effect	Status
BF16 optimizer state	Small updates rounded away	Fixed (full precision)
Checker attacks	27 of 87 attack checks accepted	Fixed (0 of 87)
Batch checking	One broken proof caused the rejection of the next proof; 15 of 40 correct proofs rejected in one test	Fixed
Action parser	Every action in a code block rejected before Lean	Fixed
Lost Lean error messages	One error type gave reward 0 instead of a penalty (186 actions)	Patch not yet verified
Proof export	Indentation of multi-line proofs changed, so exports failed	Fixed
Full-file compilation	DEV files exceeded the memory limit without a proof	Open
Prohibited-word filter	A string trick hid a prohibited command, and Lean accepted it	Fixed (version 2.2)
Run failures	Full disk at checkpoint save; missing PPO method; launch-script name conflict; wrong prompt-length count	Fixed

Some failures were engineering faults, not scientific results (Table 5).

7 Next steps

Corrected environment. Version 2.1 of the environment corrects the defects of version 1 (Figure 18). The model sees the theorem, the accepted steps, the current goals, the last Lean message, the tactics that already failed, and the remaining steps. An action contains up to four tactics, and Lean keeps all tactics before the first error, as in DeepSeek-Prover-V1.5 [11]. The reward is 1 for a proof and 0 otherwise, with $\gamma = 0.95$, so shorter proofs receive more reward. Version 2.2 corrected further faults. Integration tests with real Lean passed: 8 of 8 for version 2.1, and 86 of 86 for the version 2.2 trainer.

Curriculum ordered by proof structure. The final curriculum has five stages, ordered by the type of reasoning that a proof needs (Figure 19). All tasks come from templates in families of 8, and Lean accepts the reference proof of every task. A family that one fixed tactic mostly solves (4 or more

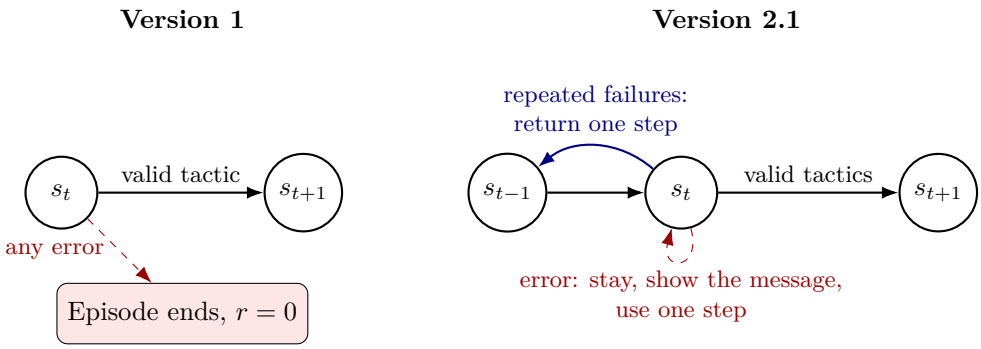


Figure 18: Treatment of a failed action in version 1 and version 2.1.

of 8) moves to S0. We defined this relabel rule after we saw the first probe results, so it was not chosen blind. Within the training set, exact duplicates were removed, which leaves 192 training tasks.

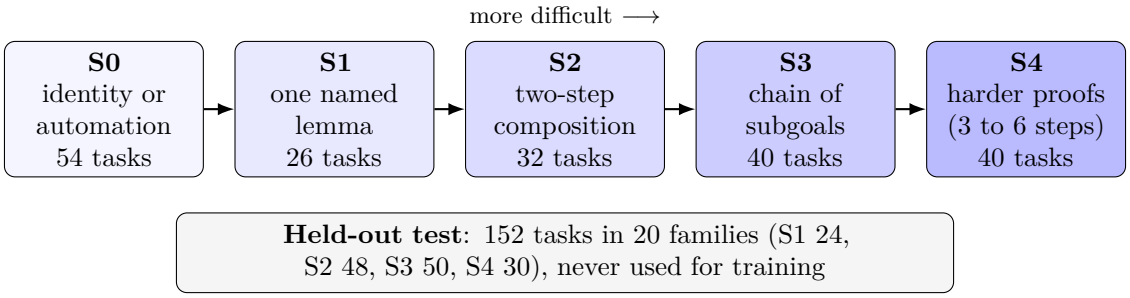


Figure 19: The final structure curriculum: 192 training tasks and 152 held-out tasks. Training excludes 670 names: every held-out, evaluation, reserve, demo, and DEV name.

Cleaning the held-out set. A semantic check compared held-out tasks with the statements behind the SFT data. In the first panel, it found 26 held-out–training pairs with the same content and 3 near-transfers; for example, `ho_coprimechain_01` restates `lean_workbook_22701`. We removed 35 tasks, which left 61 clean tasks. A second set of new families went through the same check, which removed 7 contaminated held-out families and 2 training families. We then dropped the 5 remaining held-out tasks rated "related". The final held-out set has 152 tasks: the 61 original clean tasks and 91 new ones. On the 61 original clean tasks, the SFT parent (environment version 1) solved 6, with 8 of 488 attempts accepted.

Can RL start on these stages? Before training, we ran the SFT parent 8 times on each S0 to S2 task of the first template set, with the version 1 prompt (Figure 20). With the relabelled stages, S0 had 187 of 448 attempts accepted (44 of 56 tasks mixed), S1 26 of 256 (14 of 32 mixed), and S2 10 of 256 (8 of 32 mixed). Every stage has mixed groups, so RL has a signal, but the signal is weak in S2. Under the original labels, one family that `omega` solves alone gave 32 of the 42 S2 successes, and the two real two-step families gave 8 of 64 and 0 of 64. With the version 2.1 prompt, 186 of 386 first actions of the parent failed on format. In 145 of these 186, the code block had a label other than `lean`, for example `liquorice`; no run has tested whether the label causes the failure. Therefore a short warm-start in the new format comes before RL. Its data has 1,969 training rows on 468 theorems, each traced again through the version 2.1 environment.

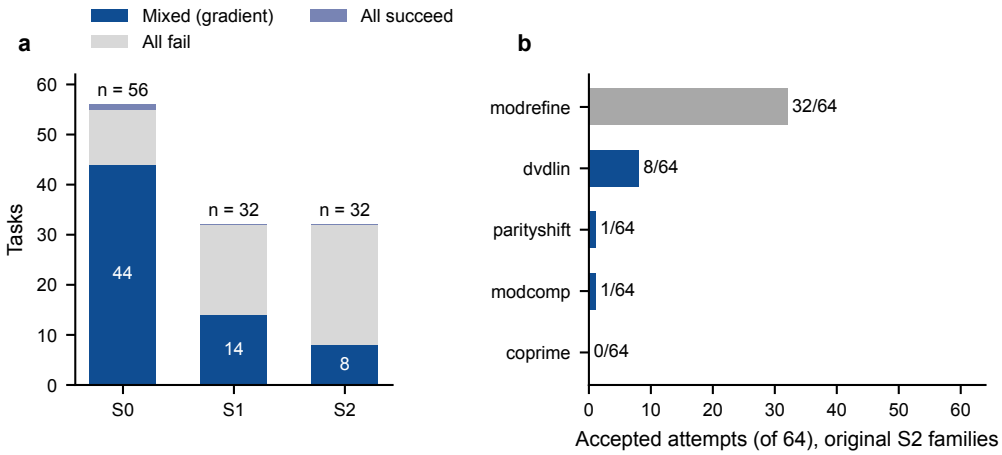


Figure 20: Readiness test of the SFT parent on the first template set, version 1 prompt, 8 attempts per task, one seed. (a) Tasks per relabelled stage by outcome. (b) Accepted attempts per family of the original S2 stage, $n = 64$; grey: the family that `omega` solves alone, later moved to S0.

The frozen plan for the final run. We recorded the protocol before the launch; no result of this run exists yet. The run tests two claims: RL improves the model over its warm-start W on held-out tasks, and a curriculum makes RL more efficient (Figure 21).

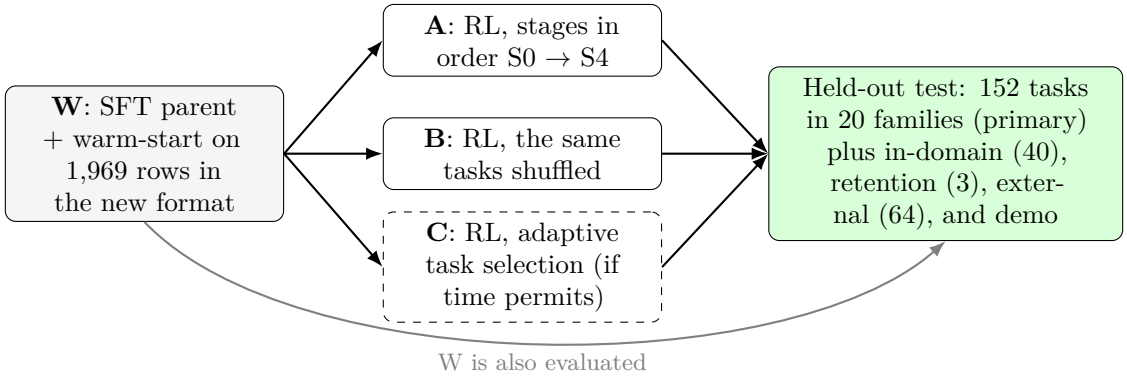


Figure 21: The frozen comparison. A and B against W test whether RL helps. A against B tests whether order helps. C tests whether tasks near a 50% success rate give more mixed groups for the same compute.

- **Settings.** Environment version 2.2, $\gamma = 0.95$, $G = 8$, at most 6 actions, REINFORCE with the leave-one-out baseline on discounted returns, learning rate 10^{-6}, seed 20261013 for every arm, at most 150 groups per arm.
- **Admission.** W samples each training task 8 times. A task enters training only if 1 to 6 of the 8 attempts succeed, so every admitted task can give mixed groups. A stage with too few admitted tasks is dropped from both arms.
- **Primary measure.** The mean fraction of the 8 attempts that Lean accepts, per held-out task. The primary test permutes whole families, because tasks in one family are correlated; Holm correction controls the two comparisons of each claim.

- **Efficiency.** For each arm, the gain in solved tasks over W per million generated tokens, together with Lean calls, updates, and the share of mixed groups.
- **Validity gates.** A against B counts as confirmatory only if both arms complete the same number of groups, with updates within 10% and censored groups within 2. The efficiency claim also needs generated tokens within 10%. If training success does not increase during RL, a null result is uninformative. No checkpoint, setting, or task is chosen from evaluation results.
- **Sample size.** In simulation, the McNemar test gave a false-positive rate of up to 0.14 against the intended 0.05; the family test stays at 0.05 or less. With the final 152 tasks, the family test detects 8 to 19 percentage points in the fraction of tasks solved, and 11 to 15 points at a baseline like the parent's. These values come from an assumed model, not from data.
- **Compute.** One L4 GPU for 12 hours, with a reservation of USD 37.01 and one attempt. A CPU fallback runs as a labelled deviation if the attempt fails.
- **Demo.** `demo_coprime` and one held-out task per level (a common divisor divides the gcd; odd m and n give an even $mn+1$; ab or $a+b$ is even), run on A if A solves at least as many held-out tasks as W, otherwise on W. Six earlier demo problems matched statements in the training or SFT data and were withdrawn.

8 Limitations

- The evaluation sets are small (3 DEV theorems in Experiment 19; 4 to 8 items in Experiment 11), and every run used one seed.
- Several sets were reused across experiments, so they now act as development sets.
- We do not know how much Lean or mathlib the model saw in pretraining.
- AI reviewers graded the natural-language answers in Experiments 9, 10, 11, and 13.
- The sample-size values come from simulations with assumed parameters.
- The numbers come from recorded run outputs; we did not repeat experiments for this report.

Sources. Each experiment has a record in `docs/EXPERIMENTS.md`. The earlier detailed report (Git commit `e3b7747`) contains every count of Experiments 1 to 13 and the full checker audit. The reward exploits are in `docs/REWARD_HACKING.md`. The script `scripts/report/make_report_figures.py` draws the data figures and names the record of each number.

References

[1] Qwen Team. Qwen2.5 technical report. https://arxiv.org/abs/2412.15115.

[2] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. https://arxiv.org/abs/2306.15626.

[3] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018.

[4] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[5] A. Ahmadian, C. Cremer, M. Gallé, et al. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. https://arxiv.org/abs/2402.14740.

[6] Z. Shao, P. Wang, Q. Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. https://arxiv.org/abs/2402.03300.

[7] Hugging Face. TRL v0.24.0 GRPO trainer documentation. https://huggingface.co/docs/trl/v0.24.0/en/grpo_trainer.

[8] A. Hatamizadeh, S. N. Akter, S. Prabhumoye, J. Kautz, et al. RLP: Reinforcement as a pretraining objective. https://arxiv.org/abs/2510.01265.

[9] H. Ying, Z. Wu, Y. Geng, et al. Lean Workbook: A large-scale Lean problem set formalized from natural language math problems. https://arxiv.org/abs/2406.03847.

[10] Training large language models for reasoning through reverse curriculum reinforcement learning. https://arxiv.org/abs/2402.05808.

[11] H. Xin et al. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. https://arxiv.org/abs/2408.08152.